

AI Use Declaration Form

Course: Introduction to Artificial Intelligence

Lab Title: Lab 3 — Neural Networks: Feedforward Networks and the Training Process

Student Name: _____ Joram E. Hanson _____

Student ID: _____ 25432028 _____

GitHub Repository Link: _____ <https://github.com/Joram-Hanson/lab-3.git> _____

Date Submitted: _____ 28/07/2026 _____

1. AI Use Summary

Question	Student Response
Did you use any AI tool for this lab?	Yes
Estimated percentage of the work influenced by AI	13 %
Did you attach evidence of AI use where applicable?	N/A

2. Details of AI Use

Complete the table below for each AI tool used. Add more rows where necessary.

Name of AI Tool Used	Purpose for Using the Tool	Prompt or Instruction Given to the Tool	Part(s) of the Work Influenced by the Tool	How I Verified, Edited, or Improved the AI Output
Claude (Anthropic)	To help me understand lecture concepts (e.g. linear regression with single neuron, backpropagation), clarify what each lab section was asking for, and explain error messages I ran into while coding.	what is the lab all about in terms of what i am suppose to do just knowledge of it explain each section Is my forward pass and backward pass in the correct order? My loss isn't decreasing. What are some	Claude was used only as a learning aid to help me understand underlying concepts, clarify technical ideas, and strengthen my conceptual understanding while working through the assignment. For the Student Reasoning questions and the Section 3 reflection, I	I ran every cell myself and used the actual outputs (loss curves, accuracy numbers, confusion matrices) from own runs to inform the reasoning answers. I fixed variable-naming and indentation bugs in my own code by identifying the lines myself

		common mistakes I should check for? Is my mathematical proof that stacked linear layers collapse into one linear layer correct? this is my section 2 Is defining self.relu1 and self.relu2 separately equivalent to calling torch.relu() inside forward()?	wrote the responses in my own words. AI was used only to help clarify the wording and to check that my explanations were coherent, grammatically correct, and accurately reflected my intended meaning. The ideas, reasoning, and final submission are my own work.	

3. Attachment of AI Output Evidence

Where required, attach evidence of AI use. Tick all that apply. Add more rows where necessary.

Evidence Type	Attached? (Yes, No, N/A)	File Name
AI-generated draft	No	
Prompt history	No	
Screenshot of AI interaction	Yes	
Exported AI conversation	No	
AI-generated code snippet	No	
Revised version showing student input	No	

4. Student Declaration

I declare that:

Declaration Statement	Response (Yes/No)
The submitted work is my own work.	Yes
Any use of AI tools has been clearly declared in this form.	Yes
The prompts, instructions, and AI-generated outputs have been disclosed where applicable.	Yes
I reviewed, tested, edited, and improved any AI-generated content before submission.	Yes
My AI usage does not exceed 25% of the entire work.	Yes
I understand that undeclared or excessive AI use may be treated as academic misconduct.	Yes

Student Signature: _____ J.E.H _____

Date: _____ 28/07/2026 _____

what is the lab all about in terms of what i am suppose to do just knowledge of it

This lab (Lab 3) is about building neural networks from the ground up, both by hand and using PyTorch, so you understand what's actually happening underneath the automation.

It's split into two main parts:

Section 1 — Build it yourself, using only NumPy

You work with a toy dataset (noisy sine wave). You start with a single neuron (just linear regression: $y = wx + b$) and manually write the gradient descent loop, computing derivatives by hand. You see it fails to fit the curve because it can only draw a straight line. Then you build a small 2-layer network by hand, including writing the forward pass and the full backward pass (backpropagation) yourself using the chain rule, no autograd. You test what happens with and without a non-linear activation (tanh), and you explore how learning rate affects training (too slow, too fast, just right).

The point of this section is to force you to understand the actual mechanics, so that when PyTorch automates it later, you know what `loss.backward()` and `optimizer.step()` are really doing.

Section 2 — Build it with PyTorch, on a real dataset

You take the same obesity-level classification dataset from Lab 2 and build a proper feedforward neural network (multi-layer perceptron) using PyTorch's tools, `nn.Module`, `autograd`, optimizers, `DataLoader`. You preprocess the data, design the network

Write a message...

+

Sonnet 5 Medium  

Claude is AI and can make mistakes. Please double-check responses.

really doing.

Section 2 — Build it with PyTorch, on a real dataset

You take the same obesity-level classification dataset from Lab 2 and build a proper feedforward neural network (multi-layer perceptron) using PyTorch's tools, `nn.Module`, `autograd`, optimizers, `DataLoader`. You preprocess the data, design the network architecture, write a real training loop, run controlled experiments (changing network size, learning rate, and dropout one at a time), evaluate the final model on a held-out test set, and compare its performance against your Lab 2 Random Forest.

Throughout both sections, every part has a "Student Reasoning" box where you have to explain, in your own words, why things happened the way they did, not just report numbers.

The overall goal: by the end, you should be able to explain what's inside a neural network's training loop, why certain design choices matter (activation functions, learning rate, regularization, feature scaling), and when a neural network is actually the right tool versus a simpler classical model.

explain each section

Orchestrated comprehensive breakdown of each component

Here's a fuller walkthrough of each section and  at it's teaching you.

Write a message...

+

Sonnet 5 Medium ▾



explain each section

Orchestrated comprehensive breakdown of each component

Here's a fuller walkthrough of each section and what it's teaching you.

Section 1 — Manual Neural Networks (NumPy only)

Part 1.1 — The single neuron

You build the simplest possible "neural network": one neuron computing $y = wx + b$, which is really just linear regression. You write gradient descent by hand, deriving the derivatives of the loss with respect to w and b yourself using the chain rule. You train it on noisy sine wave data and see it fails, the loss plateaus, and the fitted line is clearly wrong. This teaches: a straight line can never represent a curved function, no matter how well-trained it is. This is underfitting due to a hard capacity limit, not a training problem.

Part 1.2 — Hidden layers and activations

You build a small 2-layer network by hand: input $\rightarrow$ 8 hidden neurons with $\tanh \rightarrow$ 1 output. You write both the forward pass and the full backward pass (manual backpropagation) using the chain rule across two layers. You test it with $\tanh$ (it fits the curve well) and without $\tanh$ (it collapses back to a straight line). This teaches: non-linear activations are what give a network the ability to represent curves; without them, stacking linear layers is mathematically equivalent to just one linear layer.

Write a message...

+

Sonnet 5 Medium



Claude is AI and can make mistakes. Please double-check responses.

Part 1.3 — Learning rate behavior

You wrap your training loop in a function and run it with three different learning rates. You watch what happens when the rate is too small (very slow learning), too large (loss oscillates or explodes), or well-tuned (smooth convergence). This teaches: the learning rate controls how big each gradient descent step is, and there's a real trade-off, too cautious wastes time, too aggressive overshoots the minimum entirely.

Section 2 — Real Feedforward Network in PyTorch

Part 2.1 — Preprocess and split

You reload the same obesity dataset from Lab 2 and repeat the preprocessing: encoding categorical columns, scaling numeric features, and doing a stratified 60/20/20 train/validation/test split. Then you convert everything into PyTorch tensors and wrap them in `DataLoader`'s for batching. This teaches: neural networks need numeric input just like other ML models, but they're much more sensitive to feature scale because gradients scale with input magnitude.

Part 2.2 — Design the network

You define a proper `FeedForwardNet` class using PyTorch's `nn.Module`: an input layer, two hidden layers with ReLU activations, and an output layer producing 7 raw class scores (logits). You count the trainable parameters and verify $\downarrow$ math by hand. This teaches: how to structure a real neural network in PyTorch, why ReLU is commonly preferred over tanh, and how parameter count scales with layer sizes.

Write a message...



Sonnet 5 Medium ▾



My loss isn't decreasing. What are some common mistakes I should check for?

Diagnosed common training loop pitfalls and gradient computation issues

Here's a checklist of the most common culprits, roughly in order of likelihood:

1. Learning rate too small

Your code uses `lr = 0.01`. Earlier in this conversation you used `lr = 0.1` and got a clean, fast decrease from ~0.54 down to ~0.20 within 200 steps. With `lr = 0.01`, the steps are 10x smaller, so the loss might just be decreasing far more slowly, not stuck. Try printing the loss every 20 steps to check:

```
python
if step % 20 == 0:
    print(f"step {step}: loss={loss:.5f}")
```

If it's decreasing but just very gradually, that confirms it's a learning rate issue, not a bug. Bump `lr` up (e.g. back to 0.1) and see if it converges faster.

2. Forward/backward/update happening outside the loop (indentation bug)

This bit you earlier in Part 1.2, everything after `↓` first few lines silently sits outside the `for` loop due to missing indentation, so training only happens once instead of 200 times. Double

Write a message...

+

Sonnet 5 Medium  

own results, train performance keeps improving while validation performance plateaus or worsens.

Part 2.4 — Controlled experiments

You wrap the training loop into a reusable function, then run three separate experiments changing one variable at a time: network size (capacity), learning rate, and dropout (regularization). You compare results across each. This teaches: how each of these hyperparameters independently affects training and generalization, and reinforces the learning rate lessons from Part 1.3 on a real dataset.

Part 2.5 — Final evaluation and showdown

You load your best saved model, evaluate it once on the untouched test set (accuracy, macro-F1, confusion matrix), and directly compare its performance to your Lab 2 Random Forest classifier. This teaches: how to fairly evaluate a model, and critically, that neural networks aren't always the best tool, on small tabular datasets like this one, classical models often win.

Section 3 — Reflection

A set of short conceptual questions tying the whole lab together: what's really happening inside `loss.backward()` and `optimizer.step()`, which hyperparameter matters most, where you saw the worst overfitting and what fixed it, and why a plain MLP would struggle with image data (setting up the next lab, which will likely cover CNNs).

🗨️ 🔊 📄 🔄 🌐

Write a message...

+

Sonnet 5 Medium ▾ 🔊 🎧

Claude is AI and can make mistakes. Please double-check responses.

```
alpha = 0.01
axes[1].plot(x, w * x + b, color='red', label=f"fitted line: y=
{w:.2f}x + {b:.2f}")
axes[1].set_title("Single neural network fit to toy data")
axes[1].set_xlabel("x")
axes[1].set_ylabel("y")
axes[1].legend()

plt.tight_layout()
plt.show()
```

Is my forward pass and backward pass in the correct order?

Show less

Verified correct gradient descent sequence: forward, backward, update

Yes, the order is correct.

Your loop does exactly the right sequence, in this order:

1. **Forward pass** — compute `y_hat = w * x + b`, then `loss = np.mean((y_hat - y) ** 2)`
2. **Backward pass** — compute the gradients `dw` and `db` using the current `y_hat` (which is why gradients must be computed *after* the forward pass, they depend on it)
3. **Parameter update** — apply `w -= lr * dw` and `b -= lr * db`

Write a message...

+

Sonnet 5 Medium ▾



Claude is AI and can make mistakes. Please double-check responses.

what lets the network trace a curve like sine – something no single neuron could do alone.

3. Backpropagation applies the chain rule layer by layer, working backward from the output to the input, to compute how much each parameter in the network contributed to the loss.

Is my mathematical proof that stacked linear layers collapse into one linear layer correct?

Show less

11:51 PM   

Yes, your proof is correct.

Walking through it:

Starting from the two-layer computation:

$$\hat{y} = (xW_1 + b_1)W_2 + b_2$$

Distributing:

$$\hat{y} = xW_1W_2 + b_1W_2 + b_2$$

You correctly identify that since W_1, W_2, b_1, b_2 are all just fixed numbers (matrices/vectors, but fixed values once trained), the products W_1W_2 and $b_1W_2 + b_2$ are themselves just fixed matrices/vectors. So you can define:

Write a message...

+

Sonnet 5 Medium ▾

 

Claude is AI and can make mistakes. Please double-check responses.

```
model.layer1.parameters())})"
```

this is my section 2

Is defining `self.relu1` and `self.relu2` separately equivalent to calling `torch.relu()` inside `forward()`?

Show less

Yes, functionally equivalent — both apply the exact same ReLU operation (`max(0, x)`), with no learnable parameters either way, so your model's behavior and parameter count are identical regardless of which style you use.

Your approach:

```
python
self.relu1 = nn.ReLU()
...
x = self.relu1(self.layer1(x))
```

Alternative:

```
python
x = torch.relu(self.layer1(x))
```



Write a message...



Sonnet 5 Medium ▾



Claude is AI and can make mistakes. Please double-check responses.